Race Condition Prevention in xv6

Complete C implementation experiment results and synchronization analysis

Operating Systems Case Study 14
Experiment date 24 October 2026

What this case study demonstrates

Concurrent xv6 processes repeatedly allocate and release memory. Their requests meet inside the kernel physical-page allocator, whose free list is shared across CPUs. Removing its existing lock exposes unsafe concurrent updates; restoring that lock prevents simultaneous free-list updates. The user buffers are private by design. The deliberately unprotected kernel data is the shared resource where this experiment introduces the race.

The C-runner experiment completed ten original-lock attempts, ten unlocked attempts and ten restored-lock attempts. All original and restored attempts passed. Every unlocked attempt produced a failure or timeout. The normal xv6 sanity tests also passed. This report contains the executed workload, actual source changes, console output, all measured outcomes, prevention explanation, reproduction commands and complete C source appendices.

Table 1 Main results from the C runner

Condition	Attempts	PASS	Other primary outcomes
Original spinlock	10	10	None
Allocator lock removed	10	0	1 corruption, 6 panic/trap, 2 test failures, 1 timeout
Original spinlock restored	10	10	None

How to present the experiment

For the problem and solution narrative, present the unlocked race first and the restored-lock result second. Explain that a healthy original-lock baseline was measured first as a control. Standard xv6 already contains allocator synchronization; this study restores an existing mechanism rather than inventing a new lock for an originally unprotected OS.

The observed prevention method is the xv6 spinlock. Atomic operations, acquire/release memory ordering and nested local interrupt handling are components of that mechanism. Other synchronization choices are compared by suitability, not presented as additional tested implementations.

1 Scope environment and code ownership

The implementation was executed locally through Ubuntu 24.04 in WSL2 on the Windows PC. The report uses the separate run set collected by the C helper in member1_completed/c_validation. An earlier run set exists in member1_completed/evidence, but its counts are not mixed with this report.

Table 2 Fixed experimental settings

Setting	Value
xv6 revision	f780d7850765d74cbdcd1fff4b822d8829f7c9bd
Guest architecture	RISC-V under QEMU 8.2.2
Compiler	RISC-V GCC 13.3.0
QEMU configuration	4 virtual CPUs and 128 MiB RAM
Workload	4 children; 8 pages per child; 200 rounds
Allocation per child per round	8 x 4096 = 32768 bytes
Attempt protocol	Fresh QEMU guest; one stress invocation
Limits	30 s to shell; 60 s after stress command
Working checkout	/home/nithish/xv6-member1 in Ubuntu-24.04

What was written or changed

race_test.c is the C workload added to xv6. Its UPROGS entry was added to the existing Makefile. Four acquire/release statements in kernel/kalloc.c were temporarily commented out to create the unlocked variant. The original allocator was then restored byte for byte. kernel/spinlock.c and kernel/sleeplock.c are upstream xv6 implementations inspected to explain prevention; they were not newly written for this study.

member1_tools.c is the host-side C runner and packager. It builds the guest, starts QEMU, sends commands, records output, enforces host timeouts and restores the original allocator. It does not execute inside xv6 and is not the synchronization solution. The full workload and host C sources are included in the appendices and embedded as PDF attachments.

Automation-assisted execution produced these measurements. Team members should understand the code and claim only their actual review, implementation and presentation contributions.

2 Reproduction from the beginning

On this PC, Ubuntu and the toolchain were available and the checkout was prepared before the experiment. The commands below reproduce that setup in a new unused directory. Do not clone over the completed working checkout.

Windows PowerShell

```
wsl -d Ubuntu-24.04
```

Ubuntu shell

```
sudo apt install git build-essential qemu-system-misc bc \
  gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu
git clone https://github.com/mit-pdos/xv6-riscv.git ~/xv6-member1
cd ~/xv6-member1
git checkout --detach f780d7850765d74cbdcd1fff4b822d8829f7c9bd
git switch -c member1-race-study
make qemu CPUS=4
```

Inside xv6, echo hello xv6 and usertests -q were executed. The final sanity output was ALL TESTS PASSED. Ctrl+A followed by X exits QEMU and returns to Ubuntu.

Install the C workload

Copy the supplied race_test.c into user/race_test.c and add the following entry to the existing UPROGS list, retaining a tab before the entry and its trailing backslash.

```
  $U/_race_test\
```

```
make -j4 kernel/kernel fs.img
make qemu CPUS=4
```

Inside xv6 run race_test, without a leading underscore or .c extension. A separate locked smoke check passed before measured attempts.

Run the C automation

```
cc -std=c11 -Wall -Wextra -Werror -O2 member1_tools.c \
  -o /tmp/member1_tools
/tmp/member1_tools run /home/nithish/xv6-member1 \
  /home/nithish/member1-new-evidence full
```

The C runner requires the installed C test and Makefile entry, checks that the allocator matches Git HEAD, and refuses an existing output directory. It runs sanity and smoke checks, then A01-A10, B01-B10 and C01-C10. The recorded set in this report used the same full mode with c_validation as its new output directory.

3 How the C workload reaches the race

The parent opens a pipe and creates four children with fork(). Each child waits for one pipe byte. The parent sends four bytes after creation so the workers can overlap. The gate encourages concurrent work but does not guarantee simultaneous execution of particular kernel instructions.

Each child requests 32768 bytes with sbrk(), writes a distinct byte tag across all bytes, waits briefly in user space and verifies every byte. It then shrinks its heap by the same amount. Each child repeats 200 rounds. The parent checks setup errors and collects child exit statuses with wait().

```
positive sbrk()
  -> sys_sbrk() -> growproc() -> uvmalloc() -> kalloc()
negative sbrk()
  -> growproc() -> uvmdealloc() / uvmunmap() -> kfree()
```

This revision uses eager allocation through the public sbrk() wrapper. Fork, page-table setup and process exit also exercise the allocator, so failures need not occur only inside a worker buffer check.

Why this is a race between concurrent programs

The four processes have separate virtual address spaces. Their kernel requests still update the same global physical-page free list. When two CPUs execute unprotected allocator updates together, they may duplicate an allocation, lose a free page or damage list links. Equal virtual addresses in two children are normal and do not establish duplicate physical-page allocation.

The core checking logic

```
char *allocation = sbrk(bytes);
volatile uchar *memory = (volatile uchar *)allocation;
for (int offset = 0; offset < bytes; offset++)
  memory[offset] = tag;
/* After the user-space delay, verify every byte. */
for (int offset = 0; offset < bytes; offset++) {
  int actual = memory[offset];
  if (actual != tag) {
    printf("child %d: CORRUPTION round=%d offset=%d "
           "expected=%d got=%d\n",
           id, round, offset, (int)tag, actual);
    return 1;
  }
}
```

The listing above is a focused excerpt; the complete executed file, including allocation failure checks and cleanup behavior, appears in Appendix A. volatile makes reads occur again; it neither synchronizes the allocator nor fixes a race.

4 Introducing the unprotected allocator

Only four lock calls changed: acquire and release in kfree(), and acquire and release in kalloc(). All free-list operations and the stress workload stayed the same. No artificial kernel delay was introduced. The complete diff is preserved in c_validation/B-lock-removal.diff.

Unlocked critical sections actually tested

```
/* kfree(): after validation and junk fill */
r = (struct run *)pa;
// acquire(&kmem.lock);
r->next = kmem.freelist;
kmem.freelist = r;
// release(&kmem.lock);

/* kalloc() */
// acquire(&kmem.lock);
r = kmem.freelist;
if (r)
  kmem.freelist = r->next;
// release(&kmem.lock);
```

Table 3 Possible allocation interleaving without the lock

Step	CPU 0	CPU 1
1	Reads head page A	
2		Reads the same head A
3	Advances head to B	
4		Advances head to B
5	Returns A	Returns A

Both callers can receive A if both read its next pointer before later overwrites. For two frees, both CPUs can read the old head, then independently write their own page as the new head; the last store can hide the other inserted page. These are source-level explanations of possible failures, not a captured instruction-by-instruction trace.

Removing acquire/release also removes their local interrupt-state effects where no outer lock is active. The measured comparison is the original locking discipline versus its removal, not an independent test of mutual exclusion alone.

5 Actual corruption and kernel panic

B06 memory corruption

The following lines are copied exactly from the C-runner transcript. Concurrent CPUs interleave their prints, so the child label is split. The corruption tuple itself remains readable.

```
init: starting sh
$ race_test
race_test: children=4 pages=8 rounds=200
child scause=0xd sepc=0x80000aca stval=0x505050505050505
panic: 2scause=0xd sepc=0x80000aca stval=0x505050505050505
panic: kerneltrapkerneltrap
: CORRUPTION round=1 offset=16384 expected=3 got=4
race_test: pid=6 failed status=1
QEMU: Terminated
```

At round 1 and byte offset 16384, the checker expected tag 3 but read 4. That is an observed memory-content mismatch. The same attempt also contains trap and panic output. It is counted once, with CORRUPTION as its primary outcome. The log supports a mismatch; it does not prove the exact sequence of CPU instructions that caused it.

B01 kernel failure

```
$ race_test
race_test: children=4 pages=8 rounds=200
scause=0xd sepc=0x80000aca stval=0x505050505050505
panic: kerneltrap
QEMU: Terminated
```

scause=0xd is exception code 13, a load page fault. sepc records the faulting instruction address, while stval records the faulting address when reported for this fault [3]. The kernel prints panic: kerneltrap rather than completing the user workload. QEMU: Terminated is the host stopping the guest after evidence collection; it is not an xv6 success message.

Where the failing instruction maps

```
Reconstructed unlocked build with the same revision and compiler:
addr2line 0x80000aca -> kalloc, kernel/kalloc.c:76
0x80000aca: ld a5,0(s1)
C statement: kmem.freelist = r->next;
```

This mapping was obtained by rebuilding the preserved unlocked source in a separate diagnostic directory. It identifies a free-list link load in that reconstruction. The original faulting executable was not archived, so this is reconstructed address analysis, not a live debugger trace of B01. Repeated 0x05 bytes are consistent with the allocator fill pattern contaminating a pointer; that mechanism remains an inference.

6 Where a run stops and how failures are counted

B08 stopped making useful progress

```
hart 3 starting
init: starting sh
$ race_test
race_test: children=4 pages=8 rounds=200
scause=0xd sepc=0x80000aca stval=0x505050505050505
pascanic: use=0xd sepc=0x80000akca stval=0x50505050erneltrap5050505
pa
nQEMU: Terminated
```

After race_test began, this attempt produced trap diagnostics and heavily interleaved panic fragments. It did not produce a recognized complete PASS or FAIL result before the host limit. The C runner recorded TIMEOUT at 60.028 seconds and stopped QEMU. Thus the observed stopping point is after test launch and trap output, before a completed test summary. This evidence does not establish deadlock or its exact waiting state.

B07 parent reported failed workers

```
race_test: children=4 pages=8 rounds=200
cchhiilldd  31: al:l ocatCion OfRaRUiPled at TIrOoNu nrdou n1d6=
16 race_test: pidof=f5s et=81f9a2i lexepde csttead=4 gottu=s0=
1
race_test: pid=7 failed status=1
race_test: FAIL (2 child failures; setup_failed=0)
$ QEMU: Terminated
```

The parent summary reports two failed children with setup_failed=0. Earlier worker output is interleaved, so the report preserves it rather than rewriting it into a cleaner invented message. B07 is counted as TEST_FAILURE. B10 similarly ended with four child failures. Corruption text fragmented across CPUs may not match the automatic detector; the primary outcome follows the recorded summary and raw evidence remains available.

Outcome rules

Each attempt contributes one primary outcome. A corruption followed by a panic is not counted as two attempts. PASS means that invocation finished without a detected error. TEST_FAILURE includes worker or allocation failures. PANIC_OR_TRAP records a recognized kernel diagnostic. TIMEOUT records the host limit. A failure before reaching the shell would be a boot failure, although none occurred in this run set.

The helper records raw .txt output and timed .cast events as output arrives. Interleaving is preserved. A fresh guest is launched for the next attempt so the corrupted in-memory allocator state is not carried forward. The guest disk image is reused; this protocol does not promise a freshly recreated filesystem image for every boot.

7 Prevention by restoring the xv6 spinlock

The saved original allocator was restored exactly, rebuilt and tested again. Both allocating and freeing must obey the same lock discipline because they modify the same list. Protecting only one function would leave the other function able to corrupt the shared structure.

Protected critical sections actually tested

```
/* kfree(): after validation and junk fill */
r = (struct run *)pa;
acquire(&kmem.lock);
r->next = kmem.freelist;
kmem.freelist = r;
release(&kmem.lock);

/* kalloc() */
acquire(&kmem.lock);
r = kmem.freelist;
if (r)
  kmem.freelist = r->next;
release(&kmem.lock);
```

While one CPU updates the list, another CPU cannot enter a critical section protected by the same lock. The first CPU completes the whole list operation before releasing ownership. This prevents the overlapping head updates described in Section 4.

Actual restored output C01

```
$ race_test
race_test: children=4 pages=8 rounds=200
race_test: PASS (all 4 children completed)
$ QEMU: Terminated
```

All ten restored attempts produced the PASS summary. The original baseline A01-A10 also produced ten PASS summaries with identical workload settings.

Restoration checks

```
kernel/kalloc.c: OK
user/race_test.c: OK
Final allocator Git diff: empty
Final rebuild: completed successfully
```

The first two lines come from SHA-256 verification logs. The empty final diff and build logs are recorded separately. Checksums verify restoration and unchanged workload source; successful stress invocations provide workload evidence rather than a mathematical proof of the whole kernel.

8 How xv6 synchronization prevents the race

The allocator uses the upstream spinlock implementation from this fixed source revision [1]. The mechanisms below work together. They were retained or restored, not newly invented for the case study.

Atomic ownership

```
push_off();
if (holding(lk))
  panic("acquire");
while (__atomic_exchange_n(&lk->locked, 1,
                           __ATOMIC_ACQUIRE) != 0)
  ;
lk->cpu = mycpu();
```

The atomic exchange makes checking and setting lock ownership indivisible between CPUs. Only the CPU observing a previous zero acquires the lock. Contenders spin. A normal read followed by a normal write would itself be vulnerable to a race.

Memory ordering and release

```
if (!holding(lk))
  panic("release");
lk->cpu = 0;
__atomic_store_n(&lk->locked, 0, __ATOMIC_RELEASE);
pop_off();
```

Acquire ordering keeps subsequent protected accesses after lock acquisition. Release ordering makes preceding protected updates visible before another CPU acquires ownership. Atomicity and ordering are both needed; volatile or a standalone fence would not serialize the multi-statement list operation.

Local interrupt handling

push_off() disables local interrupts and tracks nesting. pop_off() restores the prior state when the outermost matching region ends. This prevents certain same-CPU interrupt re-entry and self-deadlock problems while holding a spinlock. Disabling local interrupts alone does not stop another CPU from accessing the free list.

Lock discipline

Every conflicting list operation must use the same lock. Critical sections should stay short and should not sleep while holding the spinlock. When multiple locks are needed, a consistent acquisition order helps prevent deadlock. The workload startup pipe coordinates workers; it is not the allocator race-prevention mechanism.

9 Which synchronization method is suitable

For this short, shared, multi-CPU allocator update, the existing xv6 spinlock is the appropriate demonstrated method. This recommendation follows the source context and the healthy restored runs. The study did not implement or benchmark a separate mutex, semaphore, sleep-lock or lock-free allocator, so there is no measured universal winner.

Table 4 Suitability of synchronization approaches

Method	Role or suitable use	Status in this study
xv6 spinlock	Short kernel updates that must exclude other CPUs without sleeping	Restored and tested; 10/10 restored PASS
Atomic exchange plus memory ordering	Implements spinlock ownership and ordering; not a separate whole-list algorithm	Existing components exercised through the spinlock
Local interrupt disabling	Prevents local interrupt re-entry; cannot exclude other CPUs by itself	Existing nested handling exercised through acquire/release
xv6 sleep-lock	Longer operations where a process may sleep while waiting; used for buffer ownership	Source inspected; not substituted into the allocator
Mutex or binary semaphore	Can serialize process-level resources when sleeping is permitted	Not implemented or measured here
Counting semaphore	Tracks a count of available resources; extra design needed for list updates	Not implemented or measured here
Lock-free CAS list	Requires a correct whole-list algorithm and memory-reclamation reasoning	Not implemented or measured here
volatile or a fence alone	Does not provide mutual exclusion for these list updates	Not a replacement for the allocator lock

In this revision, the sleep-lock code protects its own state with an internal spinlock, sleeps while ownership is unavailable, and wakes waiters on release. Buffer-cache code uses both spinlocks and sleep-locks for different scopes. This demonstrates why synchronization depends on context: an early-boot/short allocator path cannot be changed blindly into a sleeping operation.

Only one prevention configuration was experimentally evaluated: the original allocator spinlock discipline. The other rows are a source-based or conceptual comparison, not additional successful experimental outputs. No claims about faster performance or fairness were measured.

10 All thirty C runner attempts

All 30 attempts reached the shell and started race_test. The table excludes the separate sanity and smoke checks. Durations include a short output-collection interval after a recognized result and are not application performance measurements. B08 reached the configured host timeout.

Table 5 Per attempt primary outcomes

Attempt	Condition	Primary outcome	Seconds
A01	locked	PASS	0.857
A02	locked	PASS	0.848
A03	locked	PASS	0.847
A04	locked	PASS	0.864
A05	locked	PASS	0.843
A06	locked	PASS	0.862
A07	locked	PASS	0.841
A08	locked	PASS	0.846
A09	locked	PASS	0.855
A10	locked	PASS	0.852
B01	unlocked	PANIC_OR_TRAP	0.780
B02	unlocked	PANIC_OR_TRAP	0.779
B03	unlocked	PANIC_OR_TRAP	0.780
B04	unlocked	PANIC_OR_TRAP	0.778
B05	unlocked	PANIC_OR_TRAP	0.777
B06	unlocked	CORRUPTION	0.792
B07	unlocked	TEST_FAILURE	0.852
B08	unlocked	TIMEOUT	60.028
B09	unlocked	PANIC_OR_TRAP	0.762
B10	unlocked	TEST_FAILURE	0.865
C01	restored	PASS	0.876
C02	restored	PASS	0.852
C03	restored	PASS	0.864
C04	restored	PASS	0.886
C05	restored	PASS	0.872
C06	restored	PASS	0.868
C07	restored	PASS	0.876
C08	restored	PASS	0.889
C09	restored	PASS	0.867
C10	restored	PASS	0.859

Source: member1_completed/c_validation/results.csv. Primary outcomes preserve the C runner classification; raw transcripts retain interleaved and secondary diagnostics.

11 Conclusion and text for presentation

Conclusion

The study exposed a race in xv6 shared allocator data by removing its existing synchronization while four processes repeatedly requested and released memory on four virtual CPUs. The unlocked C-runner phase produced one corruption outcome, six panic/trap outcomes, two test failures and one timeout in ten attempts. Restoring the original allocator spinlock produced ten PASS outcomes in ten attempts, matching the ten-PASS baseline. These observations support the importance of protecting complete free-list updates with the same lock.

Problem statement for the PPT

Concurrent kernel requests can update a shared physical-page free list at overlapping times. This case study investigates the failures exposed when xv6 allocator locking is removed and compares them with the same workload after restoration.

Implementation text for the PPT

A C workload creates four processes. Each performs 200 rounds of allocating eight pages, writing a unique byte pattern, checking the stored data and freeing the memory. Three configurations are compared on four virtual CPUs: original locks, deliberately removed allocator locks and restored original locks. Ten fresh-guest attempts are recorded per configuration.

Results text for the PPT

Original locking: 10/10 PASS. Unlocked allocator: 0/10 PASS, with corruption, kernel panics, child failures and a timeout. Restored locking: 10/10 PASS. A representative run read tag 4 where tag 3 was expected. Another fault address mapped to a free-list link load in a reconstructed unlocked build.

Prevention text for the PPT

The xv6 spinlock serializes free-list updates. Atomic ownership, acquire/release ordering and nested local interrupt handling support safe critical sections. Both kalloc() and kfree() must use the same lock. For this short kernel allocator operation, the restored spinlock is suitable; other methods were not benchmarked.

Limits and future work

Timing-dependent races can pass by chance. The test does not exhaustively detect lost pages or every corruption outside worker buffers. Panics do not alone prove the exact interleaving, and a timeout does not prove deadlock. Future work could add controlled workload variations with matching locked controls, allocator invariant checks and separate instrumentation. Any such results must be labelled as new experiments.

12 Evidence references and complete source

The primary run set for this report is member1_completed/c_validation. Use its CSV, logs and recorded source variants together. Do not combine it with the earlier Python-collected run set in member1_completed/evidence. Both sets exercised the same C guest workload; the host collector changed.

Table 6 Evidence locations relative to c_validation

Evidence	File or files
All measured attempts	results.csv
Original baseline	A01-locked.txt through A10-locked.txt
Unlocked phase	B01-unlocked.txt through B10-unlocked.txt
Restored phase	C01-restored.txt through C10-restored.txt
Exact source	race_test.used.c; kalloc.locked.c; kalloc.unlocked.c; Makefile.used
Exact changes	B-lock-removal.diff
Restoration	final-restoration.diff; restoration-check.txt; final-restored-build.txt
Workload check	test.sha256; test-verification.txt
Timed console capture	Corresponding .cast file for every attempt

The guest test SHA-256 is 02211a38f38f1d92a66d9475a503eb41e54fc6c1785daa3e35fa7210558dd029. The original allocator SHA-256 is 090ff45843dadbc4e674f96967a478cea37714a62fe6b723b8ac9c34fdb15983. These values identify the sources used and restored.

Technical references

[1] MIT xv6-riscv source at revision f780d7850765d74cbdcd1fff4b822d8829f7c9bd. Consulted locally: kernel/kalloc.c, kernel/spinlock.c, kernel/sleeplock.c, kernel/bio.c, kernel/trap.c, user/ulib.c, kernel/sysproc.c, kernel/proc.c and kernel/vm.c. Repository: MIT fixed revision.

[2] Cox, Kaashoek and Morris. xv6 A simple Unix-like teaching operating system. MIT, 2025. Physical memory allocation and locking background. Official xv6 book. Source interfaces are taken from the fixed checkout when versions differ.

[3] RISC-V Ratified Specifications Library. Supervisor-Level ISA, scause and stval registers. Official supervisor specification.

Project scope: the supplied common foundation guide, four-member division and Member 1 guide in the case-study folder. Diagnostic reconstruction: /tmp/member1-diagnostic-f780-20261024 in Ubuntu; it did not modify the working kernel.

The PDF contains embedded attachments with the exact C workload, C host helper, CSV and allocator diff. PDF viewers with an attachments panel can extract them. The appendices below are readable listings. Long host-helper lines wrap for display; use the attached .c file when compiling.

Appendix A Executed xv6 C workload

race_test.c Complete source listing Line numbers are not part of the code

```
 1  // User-space stress test for the xv6-riscv physical-page allocator.
 2  // Add $U/_race_test\ to UPROGS in the xv6 Makefile.
 3  // This test detects some corruption; a PASS is not a proof of safety.
 4  #include "kernel/types.h"
 5  #include "kernel/stat.h"
 6  #include "user/user.h"
 7
 8  #define NCHILD 4
 9  #define NPAGES 8
10  #define ROUNDS 200
11  #define PAGE_BYTES 4096
12
13  static int
14  worker(int id)
15  {
16    const int bytes = NPAGES * PAGE_BYTES;
17    const uchar tag = (uchar)(id + 1);
18
19    for (int round = 0; round < ROUNDS; round++) {
20      char *allocation = sbrk(bytes);
21      if (allocation == (char *)-1) {
22        printf("child %d: allocation failed at round %d\n", id, round);
23        return 1;
24      }
25
26      // Volatile forces the verification to read memory again.
27      // It does not synchronize the kernel or fix the allocator race.
28      volatile uchar *memory = (volatile uchar *)allocation;
29      for (int offset = 0; offset < bytes; offset++)
30        memory[offset] = tag;
31
32      // User-space delay gives other workers an opportunity to overwrite
33      // an incorrectly shared page. It is not a kernel race-window delay.
34      for (volatile int delay = 0; delay < 20000; delay++)
35        ;
36
37      for (int offset = 0; offset < bytes; offset++) {
38        int actual = memory[offset];
39        if (actual != tag) {
40          printf("child %d: CORRUPTION round=%d offset=%d expected=%d got=%d\n",
41                 id, round, offset, (int)tag, actual);
42          // exit() will reclaim this child's remaining address space.
43          return 1;
44        }
45      }
46
47      if (sbrk(-bytes) == (char *)-1) {
48        printf("child %d: shrink failed at round %d\n", id, round);
49        return 1;
50      }
51    }
52    return 0;
53  }
54
55  int
56  main(void)
57  {
58    int gate[2];
59    int started = 0;
60    int setup_failed = 0;
61    int bad = 0;
62
63    printf("race_test: children=%d pages=%d rounds=%d\n",
```

Appendix A Executed xv6 C workload continued

race_test.c Complete source listing Line numbers are not part of the code

```
 64           NCHILD, NPAGES, ROUNDS);
 65
 66    if (pipe(gate) < 0) {
 67      printf("race_test: pipe setup failed\n");
 68      exit(1);
 69    }
 70
 71    for (int id = 0; id < NCHILD; id++) {
 72      int pid = fork();
 73      if (pid < 0) {
 74        printf("race_test: fork failed after %d children\n", started);
 75        setup_failed = 1;
 76        break;
 77      }
 78      if (pid == 0) {
 79        char token;
 80        close(gate[1]);
 81        // Each child waits for one byte before starting its work.
 82        int received = read(gate[0], &token, 1);
 83        close(gate[0]);
 84        if (received != 1)
 85          exit(2);
 86        exit(worker(id));
 87      }
 88      started++;
 89    }
 90
 91    close(gate[0]);
 92    if (!setup_failed) {
 93      char tokens[NCHILD];
 94      for (int i = 0; i < NCHILD; i++)
 95        tokens[i] = 'G';
 96      if (write(gate[1], tokens, NCHILD) != NCHILD) {
 97        printf("race_test: start gate failed\n");
 98        setup_failed = 1;
 99      }
100    }
101    // On setup failure, EOF lets already-created children exit.
102    close(gate[1]);
103
104    for (int i = 0; i < started; i++) {
105      int status = -1;
106      int pid = wait(&status);
107      if (pid < 0) {
108        printf("race_test: wait failed\n");
109        setup_failed = 1;
110        break;
111      }
112      if (status != 0) {
113        printf("race_test: pid=%d failed status=%d\n", pid, status);
114        bad++;
115      }
116    }
117
118    if (setup_failed || bad || started != NCHILD) {
119      printf("race_test: FAIL (%d child failures; setup_failed=%d)\n",
120             bad, setup_failed);
121      exit(1);
122    }
123
124    printf("race_test: PASS (all %d children completed)\n", NCHILD);
125    exit(0);
126  }
```

Appendix B Complete unlocked allocator

kalloc.unlocked.c Complete source listing Line numbers are not part of the code

```
 1  // Physical memory allocator, for user processes,
 2  // kernel stacks, page-table pages,
 3  // and pipe buffers. Allocates whole 4096-byte pages.
 4
 5  #include "types.h"
 6  #include "param.h"
 7  #include "memlayout.h"
 8  #include "spinlock.h"
 9  #include "riscv.h"
10  #include "defs.h"
11
12  void freerange(void *pa_start, void *pa_end);
13
14  extern char end[]; // first address after kernel.
15                     // defined by kernel.ld.
16
17  struct run {
18    struct run *next;
19  };
20
21  struct {
22    struct spinlock lock;
23    struct run *freelist;
24  } kmem;
25
26  void
27  kinit()
28  {
29    initlock(&kmem.lock, "kmem");
30    freerange(end, (void *)PHYSTOP);
31  }
32
33  void
34  freerange(void *pa_start, void *pa_end)
35  {
36    char *p;
37    p = (char *)PGROUNDUP((uint64)pa_start);
38    for (; p + PGSIZE <= (char *)pa_end; p += PGSIZE)
39      kfree(p);
40  }
41
```

Appendix B Complete unlocked allocator continued

kalloc.unlocked.c Complete source listing Line numbers are not part of the code

```
42  // Free the page of physical memory pointed at by pa,
43  // which normally should have been returned by a
44  // call to kalloc().  (The exception is when
45  // initializing the allocator; see kinit above.)
46  void
47  kfree(void *pa)
48  {
49    struct run *r;
50
51    if (((uint64)pa % PGSIZE) != 0 || (char *)pa < end || (uint64)pa >= PHYSTOP)
52      panic("kfree");
53
54    // Fill with junk to catch dangling refs.
55    memset(pa, 1, PGSIZE);
56
57    r = (struct run *)pa;
58
59    // acquire(&kmem.lock);
60    r->next = kmem.freelist;
61    kmem.freelist = r;
62    // release(&kmem.lock);
63  }
64
65  // Allocate one 4096-byte page of physical memory.
66  // Returns a pointer that the kernel can use.
67  // Returns 0 if the memory cannot be allocated.
68  void *
69  kalloc(void)
70  {
71    struct run *r;
72
73    // acquire(&kmem.lock);
74    r = kmem.freelist;
75    if (r)
76      kmem.freelist = r->next;
77    // release(&kmem.lock);
78
79    if (r)
80      memset((char *)r, 5, PGSIZE); // fill with junk
81    return (void *)r;
82  }
```

Appendix C Complete restored allocator

kalloc.locked.c Complete source listing Line numbers are not part of the code

```
 1  // Physical memory allocator, for user processes,
 2  // kernel stacks, page-table pages,
 3  // and pipe buffers. Allocates whole 4096-byte pages.
 4
 5  #include "types.h"
 6  #include "param.h"
 7  #include "memlayout.h"
 8  #include "spinlock.h"
 9  #include "riscv.h"
10  #include "defs.h"
11
12  void freerange(void *pa_start, void *pa_end);
13
14  extern char end[]; // first address after kernel.
15                     // defined by kernel.ld.
16
17  struct run {
18    struct run *next;
19  };
20
21  struct {
22    struct spinlock lock;
23    struct run *freelist;
24  } kmem;
25
26  void
27  kinit()
28  {
29    initlock(&kmem.lock, "kmem");
30    freerange(end, (void *)PHYSTOP);
31  }
32
33  void
34  freerange(void *pa_start, void *pa_end)
35  {
36    char *p;
37    p = (char *)PGROUNDUP((uint64)pa_start);
38    for (; p + PGSIZE <= (char *)pa_end; p += PGSIZE)
39      kfree(p);
40  }
41
```

Appendix C Complete restored allocator continued

kalloc.locked.c Complete source listing Line numbers are not part of the code

```
42  // Free the page of physical memory pointed at by pa,
43  // which normally should have been returned by a
44  // call to kalloc().  (The exception is when
45  // initializing the allocator; see kinit above.)
46  void
47  kfree(void *pa)
48  {
49    struct run *r;
50
51    if (((uint64)pa % PGSIZE) != 0 || (char *)pa < end || (uint64)pa >= PHYSTOP)
52      panic("kfree");
53
54    // Fill with junk to catch dangling refs.
55    memset(pa, 1, PGSIZE);
56
57    r = (struct run *)pa;
58
59    acquire(&kmem.lock);
60    r->next = kmem.freelist;
61    kmem.freelist = r;
62    release(&kmem.lock);
63  }
64
65  // Allocate one 4096-byte page of physical memory.
66  // Returns a pointer that the kernel can use.
67  // Returns 0 if the memory cannot be allocated.
68  void *
69  kalloc(void)
70  {
71    struct run *r;
72
73    acquire(&kmem.lock);
74    r = kmem.freelist;
75    if (r)
76      kmem.freelist = r->next;
77    release(&kmem.lock);
78
79    if (r)
80      memset((char *)r, 5, PGSIZE); // fill with junk
81    return (void *)r;
82  }
```

Appendix D Complete C host runner

member1_tools.c Complete source listing Line numbers are not part of the code

```
 1  /* Host-side C utility for Ubuntu/WSL. The guest program is race_test.c.
 2   * Build: cc -std=c11 -Wall -Wextra -Werror -O2 member1_tools.c -o member1_tools
 3   * run REPOSITORY NEW_EVIDENCE_DIRECTORY smoke|sanity|full
 4   * package HANDOFF_DIRECTORY
 5   * replay RAW_LOG
 6   * Uses installed make, QEMU, git, sha256sum and tar via execvp (no shell).
 7   */
 8  #define _XOPEN_SOURCE 700
 9  #define _POSIX_C_SOURCE 200809L
10  #include <dirent.h>
11  #include <errno.h>
12  #include <fcntl.h>
13  #include <limits.h>
14  #include <poll.h>
15  #include <signal.h>
16  #include <stdio.h>
17  #include <stdlib.h>
18  #include <string.h>
19  #include <sys/stat.h>
20  #include <sys/types.h>
21  #include <sys/wait.h>
22  #include <time.h>
23  #include <unistd.h>
24
25  static volatile sig_atomic_t stopped;
26  static char repository[PATH_MAX], evidence[PATH_MAX];
27  static char *original;
28  static size_t original_size;
29  static int must_restore;
30
31  static void signal_stop(int sig) { (void)sig; stopped = 1; }
32  static void fail(const char *s) { perror(s); exit(EXIT_FAILURE); }
33  static void path_join(char *out, const char *a, const char *b) {
34    if (snprintf(out, PATH_MAX, "%s/%s", a, b) >= PATH_MAX) {
35      errno = ENAMETOOLONG; fail("path");
36    }
37  }
38  static char *load(const char *path, size_t *length) {
39    FILE *f = fopen(path, "rb");
40    if (!f) fail(path);
41    if (fseek(f, 0, SEEK_END)) fail("fseek");
42    long n = ftell(f);
43    if (n < 0 || fseek(f, 0, SEEK_SET)) fail("file size");
44    char *data = malloc((size_t)n + 1);
45    if (!data) fail("malloc");
46    if (fread(data, 1, (size_t)n, f) != (size_t)n) fail("fread");
47    data[n] = 0;
48    fclose(f);
49    if (length) *length = (size_t)n;
50    return data;
51  }
52  static void save(const char *path, const char *data, size_t n) {
53    FILE *f = fopen(path, "wb");
54    if (!f) fail(path);
55    if (fwrite(data, 1, n, f) != n || fclose(f)) fail("write file");
56  }
57  static int execute(char *const args[], const char *output) {
58    pid_t pid = fork();
```

Appendix D Complete C host runner continued

member1_tools.c Complete source listing Line numbers are not part of the code

```
 59    if (pid < 0) fail("fork");
 60    if (!pid) {
 61      if (output) {
 62        int fd = open(output, O_WRONLY | O_CREAT | O_TRUNC, 0644);
 63        if (fd < 0 || dup2(fd, STDOUT_FILENO) < 0 || dup2(fd, STDERR_FILENO) < 0) _exit(126);
 64        close(fd);
 65      }
 66      execvp(args[0], args);
 67      _exit(127);
 68    }
 69    int status;
 70    while (waitpid(pid, &status, 0) < 0) if (errno != EINTR) fail("waitpid");
 71    return WIFEXITED(status) ? WEXITSTATUS(status) : 128;
 72  }
 73  static void checked(char *const args[], const char *output) {
 74    if (execute(args, output)) {
 75      fprintf(stderr, "%s failed; see %s\n", args[0], output ? output : "terminal");
 76      exit(EXIT_FAILURE);
 77    }
 78  }
 79  static void restore(void) {
 80    if (must_restore && original) {
 81      char path[PATH_MAX], log[PATH_MAX];
 82      path_join(path, repository, "kernel/kalloc.c");
 83      save(path, original, original_size);
 84      path_join(log, evidence, "final-restored-build.txt");
 85      char *args[] = {"make", "-j4", "kernel/kernel", "fs.img", NULL};
 86      if (execute(args, log)) fprintf(stderr, "Restored source; rebuild failed. See %s\n", log);
 87      must_restore = 0;
 88    }
 89  }
 90  static double seconds(void) {
 91    struct timespec t;
 92    if (clock_gettime(CLOCK_MONOTONIC, &t)) fail("clock_gettime");
 93    return (double)t.tv_sec + (double)t.tv_nsec / 1e9;
 94  }
 95  static void json_string(FILE *f, const char *s, size_t n) {
 96    fputc('"', f);
 97    for (size_t i = 0; i < n; i++) {
 98      unsigned char c = (unsigned char)s[i];
 99      if (c == '"' || c == '\\') { fputc('\\', f); fputc(c, f); }
100      else if (c < 32 || c >= 127) fprintf(f, "\\u%04x", c);
101      else fputc(c, f);
102    }
103    fputc('"', f);
104  }
105  static void csv_text(FILE *f, const char *s) {
106    fputc('"', f);
107    for (; *s; s++) { if (*s == '"') fputc('"', f); fputc(*s, f); }
108    fputc('"', f);
109  }
110  typedef struct {
111    int boot, started;
112    const char *outcome;
113    char message[512];
114    double elapsed;
115  } Result;
116  static void message_line(char *out, size_t size, const char *text, const char *needle) {
```

Appendix D Complete C host runner continued

member1_tools.c Complete source listing Line numbers are not part of the code

```
117    const char *p = strstr(text, needle);
118    if (!p) return;
119    const char *start = p;
120    while (start > text && start[-1] != '\n' && start[-1] != '\r') start--;
121    size_t n = strcspn(start, "\r\n");
122    if (n >= size) n = size - 1;
123    memcpy(out, start, n); out[n] = 0;
124  }
125  static Result guest(const char *name, const char *input, int timeout) {
126    char path[PATH_MAX], filename[128];
127    snprintf(filename, sizeof filename, "%s.txt", name);
128    path_join(path, evidence, filename);
129    FILE *log = fopen(path, "wb");
130    if (!log) fail(path);
131    snprintf(filename, sizeof filename, "%s.cast", name);
132    path_join(path, evidence, filename);
133    FILE *cast = fopen(path, "w");
134    if (!cast) fail(path);
135    fprintf(cast, "{\"version\":2,\"width\":110,\"height\":32,\"title\":");
136    json_string(cast, name, strlen(name));
137    fprintf(cast, ",\"timestamp\":%ld}\n", (long)time(NULL));
138    int to_guest[2], from_guest[2];
139    if (pipe(to_guest) || pipe(from_guest)) fail("pipe");
140    pid_t child = fork();
141    if (child < 0) fail("fork QEMU");
142    if (!child) {
143      if (dup2(to_guest[0], STDIN_FILENO) < 0 || dup2(from_guest[1], STDOUT_FILENO) < 0 ||
     dup2(from_guest[1], STDERR_FILENO) < 0) _exit(126);
144      close(to_guest[0]); close(to_guest[1]); close(from_guest[0]); close(from_guest[1]);
145      execlp("qemu-system-riscv64", "qemu-system-riscv64", "-machine", "virt", "-bios", "none",
     "-kernel", "kernel/kernel", "-m", "128M", "-smp", "4", "-nographic", "-global",
     "virtio-mmio.force-legacy=false", "-drive", "file=fs.img,if=none,format=raw,id=x0", "-device",
     "virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0", (char *)NULL);
146      _exit(127);
147    }
148    close(to_guest[0]); close(from_guest[1]);
149    Result r = {0, 0, "BOOT_FAILURE", "QEMU exited before a complete result", 0};
150    char text[262144] = "";
151    size_t used = 0;
152    double start = seconds(), test_start = 0, finish = 0, shutdown = 0;
153    int ending = 0, eof = 0;
154    while (!eof) {
155      struct pollfd fd = {from_guest[0], POLLIN, 0};
156      int ready = poll(&fd, 1, 50);
157      if (ready < 0 && errno != EINTR) fail("poll");
158      if (ready > 0 && (fd.revents & (POLLIN | POLLHUP))) {
159        char chunk[8192];
160        ssize_t n = read(from_guest[0], chunk, sizeof chunk);
161        if (n <= 0) eof = 1;
162        else {
163          fwrite(chunk, 1, (size_t)n, log); fflush(log);
164          fprintf(cast, "[%.6f,\"o\",", seconds()-start);
165          json_string(cast, chunk, (size_t)n); fputs("]\n", cast); fflush(cast);
166          if (used + (size_t)n >= sizeof text) {
167            size_t keep = sizeof text / 2;
168            memmove(text, text + used - keep, keep); used = keep;
169          }
170          memcpy(text + used, chunk, (size_t)n); used += (size_t)n; text[used] = 0;
```

Appendix D Complete C host runner continued

member1_tools.c Complete source listing Line numbers are not part of the code

```
171         }
172       }
173       double now = seconds();
174       if (!ending) {
175         if (!r.started && strstr(text, "$ ")) {
176           r.boot = r.started = 1; test_start = now;
177           size_t n = strlen(input);
178           if (write(to_guest[1], input, n) != (ssize_t)n) {
179             r.outcome = "TEST_FAILURE";
180             snprintf(r.message, sizeof r.message, "Could not send test command");
181             finish = now;
182           }
183         }
184         const char *needle = NULL;
185         if (r.started && strstr(text, "CORRUPTION round=")) { r.outcome = "CORRUPTION"; needle =
    "CORRUPTION round="; }
186         else if (strstr(text, "panic:")) { r.outcome = r.started ? "PANIC_OR_TRAP" : "BOOT_FAILURE";
     needle = "panic:"; }
187         else if (r.started && strstr(text, "race_test: FAIL (")) { r.outcome = "TEST_FAILURE";
    needle = "race_test: FAIL ("; }
188         else if (r.started && strstr(text, "race_test: PASS (")) { r.outcome = "PASS"; needle =
    "race_test: PASS ("; }
189         else if (r.started && strstr(text, "ALL TESTS PASSED")) { r.outcome = "PASS"; needle = "ALL
    TESTS PASSED"; }
190         if (needle) {
191           message_line(r.message, sizeof r.message, text, needle);
192           if (!finish) finish = now + .7;
193         }
194         if ((!r.started && now-start >= 30) || (r.started && now-test_start >= timeout)) {
195           r.outcome = r.started ? "TIMEOUT" : "BOOT_FAILURE";
196           snprintf(r.message, sizeof r.message, "Host limit reached: %d seconds", r.started ?
    timeout : 30);
197           finish = now;
198         }
199         if (stopped) { r.outcome = "INTERRUPTED"; snprintf(r.message, sizeof r.message, "Host
    interrupted the run"); finish = now; }
200         if (finish && now >= finish) {
201           r.elapsed = now - (r.started ? test_start : start);
202           const char quit[] = {1, 'x'};
203           ssize_t quit_sent = write(to_guest[1], quit, sizeof quit);
204           if (quit_sent < 0 && errno != EPIPE) perror("QEMU quit");
205           ending = 1; shutdown = now;
206         }
207       } else if (now-shutdown > 5) {
208         kill(child, SIGTERM);
209         if (now-shutdown > 7) { kill(child, SIGKILL); break; }
210       }
211     }
212     if (!ending) r.elapsed = seconds() - (r.started ? test_start : start);
213     close(to_guest[1]); close(from_guest[0]);
214     int status;
215     while (waitpid(child, &status, 0) < 0) if (errno != EINTR) fail("wait QEMU");
216     fclose(log); fclose(cast);
217     printf("%s: %s: %s (%.3fs)\n", name, r.outcome, r.message, r.elapsed); fflush(stdout);
218     return r;
219   }
220   static void build(const char *name) {
221     char path[PATH_MAX]; path_join(path, evidence, name);
```

Appendix D Complete C host runner continued

member1_tools.c Complete source listing Line numbers are not part of the code

```
222    char *args[] = {"make", "-j4", "kernel/kernel", "fs.img", NULL}; checked(args, path);
223  }
224  static void record_command(char *const args[], const char *name) {
225    char path[PATH_MAX]; path_join(path, evidence, name); checked(args, path);
226  }
227  static void run(const char *repo, const char *destination, const char *mode) {
228    if (strcmp(mode,"smoke") && strcmp(mode,"sanity") && strcmp(mode,"full")) {
229      fprintf(stderr, "Mode must be smoke, sanity, or full\n"); exit(1);
230    }
231    if (!realpath(repo, repository)) fail(repo);
232    if (mkdir(destination, 0755)) fail("New evidence directory required");
233    if (!realpath(destination, evidence) || chdir(repository)) fail("working directory");
234    char *diff[] = {"git", "diff", "--exit-code", "HEAD", "--", "kernel/kalloc.c", NULL};
235    record_command(diff, "initial-allocator.diff");
236    char allocator[PATH_MAX], path[PATH_MAX]; path_join(allocator, repository, "kernel/kalloc.c");
237    original = load(allocator, &original_size);
238    path_join(path, evidence, "kalloc.locked.c"); save(path, original, original_size);
239    must_restore = 1; atexit(restore);
240    char *rev[] = {"git", "rev-parse", "HEAD", NULL}; record_command(rev, "revision.txt");
241    char *qver[] = {"qemu-system-riscv64", "--version", NULL}; record_command(qver,
     "qemu-version.txt");
242    char *cver[] = {"riscv64-linux-gnu-gcc", "--version", NULL}; record_command(cver,
     "compiler-version.txt");
243    char *hash[] = {"sha256sum", "user/race_test.c", NULL}; record_command(hash, "test.sha256");
244    char *alloc_hash[] = {"sha256sum", "kernel/kalloc.c", NULL}; record_command(alloc_hash,
     "allocator-original.sha256");
245    path_join(path, repository, "user/race_test.c"); size_t n;
246    char *test = load(path, &n); path_join(path, evidence, "race_test.used.c"); save(path, test, n);
      free(test);
247    path_join(path, repository, "Makefile"); char *mk = load(path, &n);
248    if (!strstr(mk, "$U/_race_test")) { fprintf(stderr,"Add the test to UPROGS first.\n"); exit(1);
     }
249    path_join(path, evidence, "Makefile.used"); save(path, mk, n); free(mk);
250    path_join(path, evidence, "protocol.txt");
251    const char *protocol = "Host automation: C; guest test: C\nCPUs=4 RAM=128MiB children=4 pages=8
     rounds=200\nFresh guest per attempt; boot limit=30s; stress limit=60s; no kernel timing
     instrumentation.\nNew C-tool runs are separate from the original 30 recorded attempts.\n";
252    save(path, protocol, strlen(protocol));
253    build("initial-build.txt");
254    if (!strcmp(mode,"sanity") || !strcmp(mode,"full")) {
255      Result sanity = guest("00-normal-sanity", "echo hello xv6\nusertests -q\n", 240);
256      if (strcmp(sanity.outcome,"PASS") || stopped) exit(1);
257      if (!strcmp(mode,"sanity")) { restore(); return; }
258    }
259    Result smoke = guest("01-locked-smoke", "race_test\n", 60);
260    if (strcmp(smoke.outcome,"PASS") || stopped) exit(1);
261    if (!strcmp(mode,"smoke")) { restore(); return; }
262    path_join(path, evidence, "results.csv"); FILE *csv = fopen(path,"w"); if (!csv) fail(path);
263    fputs("attempt_id,phase,commit,cpus,children,pages_per_child,rounds,boot_reached_shell,test_star
     ted,outcome,elapsed_seconds,exact_message,transcript,recording,notes\n", csv);
264    path_join(path, evidence, "revision.txt"); char *commit = load(path,NULL);
     commit[strcspn(commit,"\r\n")] = 0;
265    const char *phases[] = {"locked","unlocked","restored"};
266    for (int phase=0; phase<3 && !stopped; phase++) {
267      if (phase == 1) {
268        char *variant = malloc(original_size + 64); if (!variant) fail("malloc");
269        size_t dest=0; int edits=0;
270        for (size_t i=0; i<original_size;) {
```

Appendix D Complete C host runner continued

member1_tools.c Complete source listing Line numbers are not part of the code

```
271         if (!strncmp(original+i,"  acquire(&kmem.lock);",22) || !strncmp(original+i,"
    release(&kmem.lock);",22)) {
272           memcpy(variant+dest,"  // ",5); dest+=5; i+=2; edits++;
273         } else variant[dest++] = original[i++];
274       }
275       if (edits != 4) { fprintf(stderr,"Expected exactly four allocator lock calls; got
    %d\n",edits); free(variant); exit(1); }
276       save(allocator,variant,dest); path_join(path,evidence,"kalloc.unlocked.c");
    save(path,variant,dest); free(variant);
277       char *args[] = {"git","diff","--","kernel/kalloc.c",NULL};
    record_command(args,"B-lock-removal.diff");
278     } else if (phase == 2) save(allocator, original, original_size);
279     char filename[128]; snprintf(filename,sizeof filename,"%c-build.txt",'A'+phase);
    build(filename);
280     char *check[] = {"sha256sum","-c",NULL,NULL}; path_join(path,evidence,"test.sha256");
    check[2]=path;
281     char check_log[PATH_MAX]; path_join(check_log,evidence,"test-verification.txt");
    checked(check,check_log);
282     for (int i=1; i<=10 && !stopped; i++) {
283       char name[80]; snprintf(name,sizeof name,"%c%02d-%s",'A'+phase,i,phases[phase]);
284       Result r = guest(name,"race_test\n",60);
285       fprintf(csv,"%c%02d,%s,%s,4,4,8,200,%s,%s,%s,%.3f,",'A'+phase,i,phases[phase],commit,r.boot?
    "True":"False",r.started?"True":"False",r.outcome,r.elapsed);
286       csv_text(csv,r.message); fprintf(csv,",%s.txt,%s.cast,C host automation; fresh guest;
    uninstrumented\n",name,name); fflush(csv);
287       if (phase==0 && strcmp(r.outcome,"PASS")) { fclose(csv); free(commit); exit(1); }
288     }
289   }
290   fclose(csv); free(commit);
291   restore();
292   char *final_diff[] = {"git","diff","--exit-code","HEAD","--","kernel/kalloc.c",NULL};
    record_command(final_diff,"final-restoration.diff");
293   char *verify[] = {"sha256sum","-c",NULL,NULL};
    path_join(path,evidence,"allocator-original.sha256"); verify[2]=path;
294   char check_log[PATH_MAX]; path_join(check_log,evidence,"restoration-check.txt");
    checked(verify,check_log);
295   if (stopped) exit(1);
296 }
297
298 /* Small RFC4180-style field reader for the recorded results. */
299 static int csv_row(FILE *f, char fields[][2048], int capacity) {
300   int c=fgetc(f), field=0, quoted=0;
301   size_t n=0;
302   if (c==EOF) return 0;
303   for (;;) {
304     if (field>=capacity || n>=2047) { fprintf(stderr,"CSV field too long\n"); exit(1); }
305     if (c=='"') {
306       if (quoted) {
307         c=fgetc(f);
308         if (c=='"') { fields[field][n++]='"'; c=fgetc(f); continue; }
309         quoted=0; continue;
310       }
311       if (n==0) quoted=1; else fields[field][n++]=(char)c;
312     } else if ((c==',' || c=='\n' || c==EOF) && !quoted) {
313       fields[field++][n]=0; n=0;
314       if (c!=',') return field;
315     } else if (c!='\r') fields[field][n++]=(char)c;
316     c=fgetc(f);
```

Appendix D Complete C host runner continued

member1_tools.c Complete source listing Line numbers are not part of the code

```
317      if (c==EOF && quoted) { fprintf(stderr,"Unclosed CSV quote\n"); exit(1); }
318    }
319  }
320  static int compare_names(const void *a, const void *b) { return strcmp(*(char *const *)a,*(char
     *const *)b); }
321  static void manifest_tree(FILE *manifest, const char *base, const char *relative) {
322    char directory[PATH_MAX]; path_join(directory,base,relative);
323    DIR *d=opendir(directory); if (!d) fail(directory);
324    char **names=NULL; size_t count=0; struct dirent *entry;
325    while ((entry=readdir(d))) {
326      if (!strcmp(entry->d_name,".") || !strcmp(entry->d_name,"..") ||
     !strcmp(entry->d_name,"SHA256SUMS.txt")) continue;
327      char **next=realloc(names,(count+1)*sizeof *names); if (!next) fail("realloc"); names=next;
328      names[count]=strdup(entry->d_name); if (!names[count++]) fail("strdup");
329    }
330    closedir(d); qsort(names,count,sizeof *names,compare_names);
331    for (size_t i=0;i<count;i++) {
332      char rel[PATH_MAX],full[PATH_MAX];
333      if (*relative) path_join(rel,relative,names[i]); else snprintf(rel,sizeof rel,"%s",names[i]);
334      path_join(full,base,rel); struct stat st; if (lstat(full,&st)) fail(full);
335      if (S_ISDIR(st.st_mode)) manifest_tree(manifest,base,rel);
336      else if (S_ISREG(st.st_mode)) {
337        int fd[2]; if (pipe(fd)) fail("hash pipe"); pid_t p=fork(); if (p<0) fail("hash fork");
338        if (!p) { if (dup2(fd[1],STDOUT_FILENO)<0) _exit(126); close(fd[0]); close(fd[1]);
     execlp("sha256sum","sha256sum",full,(char*)NULL); _exit(127); }
339        close(fd[1]); FILE *hash=fdopen(fd[0],"r"); if (!hash) fail("fdopen"); char sum[65];
340        if (fscanf(hash,"%64s",sum)!=1) { fprintf(stderr,"Hash failed\n"); exit(1); }
341        while (fgetc(hash)!=EOF) {} fclose(hash); int status;
342        while(waitpid(p,&status,0)<0) if(errno!=EINTR) fail("wait hash");
343        if (!WIFEXITED(status) || WEXITSTATUS(status)) exit(1);
344        fprintf(manifest,"%s  %s\n",sum,rel);
345      }
346      free(names[i]);
347    }
348    free(names);
349  }
350  static void package(const char *folder) {
351    char base[PATH_MAX],path[PATH_MAX]; if (!realpath(folder,base)) fail(folder);
352    path_join(path,base,"evidence/results.csv");
353    const char *relative_evidence = "evidence";
354    if (access(path,R_OK)) { relative_evidence = ""; path_join(path,base,"results.csv"); }
355    FILE *csv=fopen(path,"r"); if (!csv) fail(path);
356    char rows[31][15][2048]; int n=0;
357    if (csv_row(csv,rows[30],15)!=15) { fprintf(stderr,"Unexpected CSV header\n"); exit(1); }
358    int fields;
359    while ((fields=csv_row(csv,rows[n],15))) {
360      if (fields!=15 || n>=30) { fprintf(stderr,"Expected exactly 30 result rows\n"); exit(1); }
361      n++;
362    }
363    fclose(csv);
364    if (n!=30) { fprintf(stderr,"Expected 30 attempts; found %d\n",n); exit(1); }
365    int counts[3][5]={{0}},started[3]={0},boot[3]={0};
366    const char *phase_names[]={"locked","unlocked","restored"};
367    const char *outcomes[]={"PASS","CORRUPTION","PANIC_OR_TRAP","TEST_FAILURE","TIMEOUT"};
368    for (int i=0;i<n;i++) {
369      int phase=-1, outcome=-1;
370      for(int j=0;j<3;j++) if(!strcmp(rows[i][1],phase_names[j])) phase=j;
371      for(int j=0;j<5;j++) if(!strcmp(rows[i][9],outcomes[j])) outcome=j;
```

Appendix D Complete C host runner continued

member1_tools.c Complete source listing Line numbers are not part of the code

```
372      if(phase<0 || outcome<0) { fprintf(stderr,"Unknown phase/outcome\n"); exit(1); }
373      for(int j=0;j<i;j++) if(!strcmp(rows[i][0],rows[j][0])) { fprintf(stderr,"Duplicate
     attempt\n"); exit(1); }
374      counts[phase][outcome]++; boot[phase]+=!strcmp(rows[i][7],"True");
     started[phase]+=!strcmp(rows[i][8],"True");
375      for(int j=12;j<=13;j++) {
376        char rel[PATH_MAX];
377        if (*relative_evidence) path_join(rel,relative_evidence,rows[i][j]);
378        else snprintf(rel,sizeof rel,"%s",rows[i][j]);
379        path_join(path,base,rel); if(access(path,R_OK)) fail(path);
380      }
381    }
382    path_join(path,base,"C_RESULTS_SUMMARY.md"); FILE *report=fopen(path,"w"); if(!report)
     fail(path);
383    fputs("# Results checked and packaged by C\n\n",report);
384    fputs(*relative_evidence ? "Original observations are preserved. They were collected before the
     helper-language conversion; the C runner has its own separate validation directory.\n\n" : "Fresh
     observations collected by the C runner. This is a separate run set; do not merge with the original
      30 attempts.\n\n",report);
385    fputs("| Phase | Attempts | Shell reached | Test started | PASS | Corruption | Panic/trap | Test
      failure | Timeout |\n|---|---:|---:|---:|---:|---:|---:|---:|---:|\n",report);
386    for(int i=0;i<3;i++) {
387      int total=0; for(int j=0;j<5;j++) total+=counts[i][j];
388      if(total!=10) { fprintf(stderr,"Expected 10 attempts per phase\n"); fclose(report); exit(1); }
389      fprintf(report,"| %s | %d | %d | %d",phase_names[i],total,boot[i],started[i]);
390      for(int j=0;j<5;j++) fprintf(report," | %d",counts[i][j]);
391      fputs(" |\n",report);
392    }
393    fputs("\nA PASS is an observation under this workload, not proof of absence of races. Some
     output is interleaved across CPUs. See CSV notes and raw logs.\n",report);
394    if (*relative_evidence) fputs("\nIn the original run set B10 includes trap diagnostics before
     the host timeout; it remains counted once as TIMEOUT.\n",report);
395    fputs("\n## Every recorded attempt\n\n| Attempt | Outcome | Seconds | Raw log
     |\n|---|---|---:|---|\n",report);
396    for(int i=0;i<n;i++) fprintf(report,"| %s | %s | %s | [log](%s%s%s)
     |\n",rows[i][0],rows[i][9],rows[i][10],relative_evidence,*relative_evidence?"/":"",rows[i][12]);
397    fclose(report);
398    path_join(path,base,"SHA256SUMS.txt"); FILE *manifest=fopen(path,"w"); if(!manifest) fail(path);
399    manifest_tree(manifest,base,""); fclose(manifest);
400    char archive[PATH_MAX];
401    if(snprintf(archive,sizeof archive,"%s.tar.gz",base)>=(int)sizeof archive) fail("archive path");
402    char *args[]={"tar","-czf",archive,"-C",base,".",NULL}; checked(args,NULL);
403    printf("C package complete: 30 unique attempts, 10 per phase.\n%s\n",archive);
404  }
405  int main(int argc,char **argv) {
406    signal(SIGINT,signal_stop); signal(SIGTERM,signal_stop); signal(SIGPIPE,SIG_IGN);
407    if(argc==5 && !strcmp(argv[1],"run")) run(argv[2],argv[3],argv[4]);
408    else if(argc==3 && !strcmp(argv[1],"package")) package(argv[2]);
409    else if(argc==3 && !strcmp(argv[1],"replay")) {
410      size_t n; char *text=load(argv[2],&n); if(fwrite(text,1,n,stdout)!=n) fail("stdout");
     free(text);
411    } else {
412      fprintf(stderr,"Usage:\n  %s run REPO NEW_EVIDENCE_DIR smoke|sanity|full\n  %s package
     HANDOFF_DIR\n  %s replay RAW_LOG\n",argv[0],argv[0],argv[0]); return 1;
413    }
414    free(original); original=NULL;
415    return 0;
416  }
```